

Introducción a la Ciencia de Datos

Guía de trabajos prácticos N° 8

Tabla de contenidos

Introducción al modelado de datos. Modelo de regresión lineal	1
Parte 1. Modelo lineal simple	2
Parte 2. Modelo lineal múltiple	9
Bibliografía obligatoria	13

Introducción al modelado de datos. Modelo de regresión lineal

En esta guía, vamos a dar los primeros pasos en el vasto mundo del **modelado de datos**. Este es un paso clave en el desarrollo de un proyecto de ciencia de datos, y una herramienta que seguirán perfeccionando a lo largo de toda la carrera.

Modelar datos sirve principalmente para cuatro cosas:

1. **Cuantificar** (es decir, ponerle números) a un patrón o una relación que hayamos encontrado a través de un análisis exploratorio de datos.
2. **Explorar** patrones en los datos. Podemos usar modelos para quitar las tendencias más obvias de un conjunto de datos y explorar la existencia de patrones más sutiles.
3. **Resumir** la información presente en un conjunto de datos para poder comunicar las conclusiones más fácilmente.
4. **Predecir** valores de observaciones que no tenemos.

Para modelar datos, lo primero que necesitamos es especificar qué queremos modelar. En el caso de modelos supervisados, como los que vamos a ver acá, esto significa especificar qué variable (o variables; puede ser más de una) queremos usar como *objetivo* para que nuestro modelo describa. El segundo paso, es definir un modelo. En su visión más simple, esto no es más que una expresión matemática que vincula las variables que no son el *objetivo*, que llamaremos variables explicativas o predictoras, o covariables, pero que tienen muchos [otros nombres](#).

En esta materia solo veremos modelos de *regresión*. Es decir, modelos en los que la variable *objetivo* es continua. No discutiremos los modelos de *clasificación*, que son aquellos en los que la variable *objetivo* es discreta. Para entrar a este mundo, vamos a estudiar una familia de modelos muy importante, la más usada, y que tiene gran utilidad: los modelos lineales. En particular, hoy vamos a ver **modelos lineales simples y múltiples**.

En R estos modelos se implementan en el paquete `stats`, en la función `lm` (de *linear model*). Además de esto, en esta guía mostramos el uso de varias funciones prácticas del paquete `modelr`.

Empecemos entonces, cargando `modelr` (`stats` se carga automáticamente) y nuestro querido `tidyverse`.

```
library(tidyverse)
library(modelr)
```

Parte 1. Modelo lineal simple

Lo básico

Volvamos un *dataset* de la primera clase: iris. Recuerden que está incluido en el paquete `datasets` de R, que se carga automáticamente. De manera que al abrir **RStudio** ya debería estar cargado.

Pueden ver algunas características de los datos con la función `summary`.

```
summary(iris)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300
Median :5.800	Median :3.000	Median :4.350	Median :1.300

Mean	:5.843	Mean	:3.057	Mean	:3.758	Mean	:1.199
3rd Qu.	:6.400	3rd Qu.	:3.300	3rd Qu.	:5.100	3rd Qu.	:1.800
Max.	:7.900	Max.	:4.400	Max.	:6.900	Max.	:2.500
Species							
setosa	:50						
versicolor	:50						
virginica	:50						

Para esta primera experiencia, vamos a estudiar cómo varía el ancho de pétalos de flores de la especie Versicolor con respecto a su largo.

1. Filtren el *dataset* para quedarse solo con los datos de Versicolor; guarden el resultado en una variable llamada `df`. Hagan un gráfico del ancho de los pétalos en función de su largo.

Observen la tendencia que existe entre ambas variables. Parece razonable ajustar estos datos con una recta. Para eso, hacemos uso de la función `lm`. Esta función usa una mini sintaxis particular para definir los modelos a ajustar. En este caso, tenemos que decirle cuál es nuestra variable *objetivo* (`Petal.Width`) y cuál es la variable explicativa (`Petal.Length`). Esto en `lm` se expresa así usando la virgulilla de la ñ:

```
lm(Petal.Width ~ Petal.Length, data=df)
```

Noten que además de la fórmula, hay que pasar el *dataset* que usamos para el ajuste en el argumento `data`. Con esta sintaxis, lo que estamos haciendo es ajustar un modelo de la forma:

$$\text{Petal.Width} = a + b \cdot \text{Petal.Length}$$

La tarea de la función `lm` es encontrar el valor de los **parámetros** `a` y `b` que “mejor ajustan” los datos.

2. Corran el código de arriba y guarden el resultado en la variable `mod` (de modelo), que vamos a usar más adelante.

```
mod <- lm(Petal.Width ~ Petal.Length, data=df)
```

Lo primero que podemos hacer con un modelo ajustado es usar la función `summary`. (**Nota:** muchos objetos de R pueden usarse como argumentos de `summary`, que se adapta a cada uno. En este caso, nos da mucha información sobre el modelo).

```
summary(mod)
```

Call:

```
lm(formula = Petal.Width ~ Petal.Length, data = df)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.273031	-0.073373	-0.006479	0.086271	0.295231

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.08429	0.16070	-0.525	0.602
Petal.Length	0.33105	0.03750	8.828	1.27e-11 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1234 on 48 degrees of freedom

Multiple R-squared: 0.6188, Adjusted R-squared: 0.6109

F-statistic: 77.93 on 1 and 48 DF, p-value: 1.272e-11

¡Acá hay muchísima información! Concentrémonos en los valores de los coeficientes encontrados por la función (también pueden obtenerse con `coef(mod)`). Hay toda una sección dedicada a esto:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.08429	0.1607	-0.5245	0.60234
Petal.Length	0.33105	0.0375	8.8280	0.00000

En esta sección, tenemos una fila por parámetro, y para cada parámetro tenemos cuatro valores y, eventualmente, un simbolito:

- **Estimate:** el valor del parámetro que mejor ajusta los datos.
- **Std. Error:** el desvío del estimador. Es decir, el rango dentro del cual se puede mover el valor de los parámetros sin que el ajuste cambie mucho (estamos siendo, a propósito, muy poco formales con estas definiciones, que ya van a ver con lujo de detalles en otras materias).

- **t value**: el **valor t**, que no es más que el cociente entre el valor del estimador y su desvío (pruébenlo).
- **Pr(>|t|)**: un **p valor** asociado con este **valor t**, que nos da una probabilidad. ¿Probabilidad de qué? Ah, es un trabalenguas, pero la respuesta es “la probabilidad de que, si no existe una relación entre ambas variables, hayamos obtenido este **valor t** (o uno más extremo) por azar”.
- Por último, tenemos los códigos de significancia, que aparecen como estrellitas (***) , lo que significa que es un parámetro muy significativo. En otras palabras, sería rarísimo que, si la relación entre ambas variables no existiera, obtuviéramos un valor así de alto de la pendiente solo por variaciones estadísticas de los datos. En otras palabras, podemos afirmar que esa relación existe.

En este caso, vemos que la pendiente tiene tres estrellitas (***), lo que significa que es un parámetro muy significativo. En otras palabras, sería rarísimo que, si la relación entre ambas variables no existiera, obtuviéramos un valor así de alto de la pendiente solo por variaciones estadísticas de los datos. En otras palabras, podemos afirmar que esa relación existe.

Además, el número nos dice algo importante:

Por cada centímetro que aumenta el largo del pétalo (`Petal.Length`) el ancho del pétalo (`Petal.Width`) aumenta, **en promedio** 0.331 cm.

! Importante

Es clave remarcar que este aumento es **en promedio**, porque no hay una relación uno a uno entre ancho y largo del pétalo, sino que hay una *dispersión* en torno a este valor medio.

! Importante 2

En este modelo, este aumento es igual para cualquier valor del largo del pétalo; es decir, es lo mismo si estamos en el rango de 1 cm a 2 cm que de 5 cm a 6 cm; por ser *lineal* el modelo predice el mismo aumento en el ancho del pétalo.

Las predicciones

Lo primero que uno quiere hacer es ver la recta que recién ajustamos. Para eso, podemos usar la función `modelr::add_predictions`, que toma un *dataset* y un modelo ajustado y calcula las predicciones del modelo. El resultado aparece en una nueva variable, llamada por defecto `pred`. Corran el código de abajo para agregar las predicciones al *dataset*.

```
df <- df %>% add_predictions(model=mod)
```

Nota

Por supuesto, también podríamos haber hecho a mano el cálculo de las predicciones, tomando el valor de los coeficientes e implementando la fórmula del modelo en un `mutate`, pero esta manera es mucho más general, como vamos a ver más adelante.

3. Creen un gráfico con los datos que ajustaron como puntos y agreguen las predicciones de la variable `pred` como puntos rojos. Usen después `geom_line` sobre esta misma variable para dibujar una recta que represente la curva que ajustamos.
4. Seguramente usar tantos puntos para generar una recta es demasiado. Usen `geom_abline` (lean la docu, de ser necesario) y los valores de los coeficientes para hacer el mismo gráfico sin `geom_line`.

Tip

Pueden usar `coef(mod)[['(Intercept)']]` y `coef(mod)[['Petal.Length']]` para acceder a la ordenada al origen y la pendiente.

Importante

Fíjense que `geom_abline` solo nos sirve si el modelo es una recta. Sin embargo, siempre se puede usar `add_predictions` **independientemente de qué modelo estemos usando**.

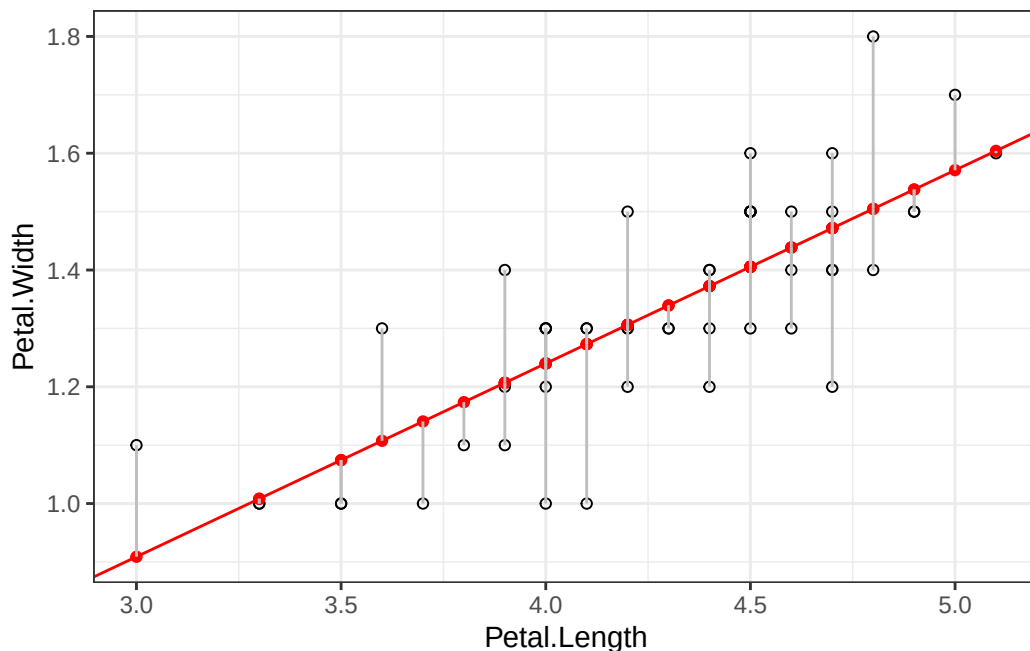
Evaluación del modelo

Una parte importantísima del trabajo con modelos es evaluarlos. Para eso, podemos usar varias métricas. En la salida del `summary` aparecen dos que vamos a discutir:

- El error estándar de los residuos (`Residual standard error`).
- El R^2 (`Multiple R-squared`).

Pero para esto necesitamos primero entender qué son los residuos. El residuo de una observación (*data point*) es la distancia entre el valor de la variable *objetivo* de esa observación, y la predicción del modelo para ese punto (es decir, el modelo evaluado en

las variables predictoras para ese punto). Gráficamente, podemos ver los residuos como el largo de los segmentos que unen la curva del modelo con los datos:



A cada punto, le corresponde un residuo, ya sea negativo o positivo. Un gráfico que es muy importante para estudiar el comportamiento del modelo es el gráfico de los residuos:

5. Calculen los residuos como la diferencia entre cada punto y el modelo. Usen `mutate` para crear una variable nueva e incluirla en el *dataset*. Hagan un gráfico de los residuos en función del valor predicho (`pred`). Agreguen una línea horizontal en cero, para evaluar el modelo. Lo deseable es que los residuos se distribuyan de manera uniforme arriba y abajo de la recta horizontal. ¿Es así? Si no, ¿qué patrón ven en esta gráfica?

💡 Tip

Los residuos también pueden calcularse y agregarse a un *data frame* con la función `modelr::add_residuals`. De nuevo, esta función puede usarse siempre, independientemente del modelo que usemos.

La primera métrica da precisamente una idea de **cuánto se dispersan los residuos alrededor del modelo**. Se calcula como la raíz del promedio de los residuos al cuadrado,

pero ajustando por la cantidad de parámetros en el modelo:

$$RMSE = \sqrt{\frac{1}{N-2} \sum_i r_i^2},$$

donde r_i es el residuo del dato i y N es la cantidad de puntos. Aparece $N - 2$ porque tenemos dos parámetros en este modelo.

Vuelvan a ver la salida de `summary(mod)`. Van a ver que además de esta métrica, tenemos, al principio, los cinco números mágicos (mínimo, máximo, mediana, 1er y 3er cuartiles) de la distribución de los residuos. Acá los calculamos de nuevo usando la función `fivenum()`:

```
cat(round(fivenum(mod$residuals), 6))
```

```
-0.273031 -0.073715 -0.006479 0.094547 0.295231
```

Nota

Si comparan con detalle, verán que los extremos y la mediana coinciden, pero los cuartiles no. Esto se debe a una diferencia en la manera de calcular el Q1 y Q3 entre ambas funciones. No hay un acuerdo universal de cómo calcular estos valores. Pueden ver una discusión detallada en este [blog](#).

6. Usen `modelr::add_residuals` para agregar los residuos al *data frame* que venimos usando, y después usen `summarise` para calcular el valor del RMSE. Comparen con la salida del `summary` que aparece más arriba.

El R^2 es otra métrica comúnmente usada para evaluar modelos; de alguna manera, **representa la fracción de la dispersión de los datos que es capturada por el modelo**. Su valor máximo es 1, cuando el modelo pasa por todos los puntos.

Puede calcularse con la función de `modelr::rsquare`:

```
rsquare(model=mod, data=df)
```

```
[1] 0.6188467
```

7. R base cuenta con la función `plot`, que al igual que `summarise` puede usarse con distintos objetos. Vean qué pasa cuando se usa con un modelo y con un *data frame*.

Parte 2. Modelo lineal múltiple

Dos variables continuas

Por supuesto, no es necesario usar una única variable para intentar modelar la variable *objetivo*. Si tenemos más de una variable explicativa, podemos usar ambas y entrarlas en la fórmula de `lm` como $y \sim x_1 + x_2$, por ejemplo.

! Importante

Ojo que esto no significa que las variables se suman directamente. En la **mini-sintaxis de `lm`**, esto significa un modelo de la forma

$$y = a + b \cdot x_1 + c \cdot x_2 \text{ .}$$

8. Elijan alguna de las otras variables del *dataset* de iris (filtrando solo Versicolor todavía) como variable explicativa adicional. Usen gráficos de los residuos en función de estas variables para intentar elegir una.

💡 Tip

No hay una respuesta clara, ninguna de las variables es muy interesante, pero una tiene un atisbo de algo mejor.

9. Usen `lm` para ajustar el modelo con esa variable adicional (llámenlo `mod2`) y vean cómo cambian los parámetros y cuáles son significativos. ¿Cómo cambia el R^2 y la dispersión de los residuos?

En base a estos resultados, seguro estamos tentados a elegir un modelo sobre otro. Pero la cosa es mucho más complicada, lamentablemente. Cuantos más parámetros tenga el modelo, mejor reproducirá los datos, independientemente de qué parámetro usemos para las variables predictoras. En la Guía 10 veremos formas más rigurosas de **comparar y elegir modelos**.

10. Usen las funciones de `modelr` para calcular las predicciones de este nuevo modelo, sus residuos y realizar un gráfico similar al de los puntos 3 y 4. ¿Qué problema encuentran?

El problema para hacer este tipo de gráfico es que para cada valor de la primera variable predictora (`Petal.Length`), el modelo toma una infinidad de valores, dependiendo del valor de la segunda variable predictora. Una posible solución es hacer una recta para cada valor de una serie de valores de la segunda variable.

Si tienen nombres estándares para todo, prueben este código para realizar un plot para cinco valores de la segunda variable predictora (aquí `Sepal.Width`, pero tal vez ustedes usaron otro valor). Vamos a crear una grilla de valores y calcular las predicciones sobre esa grilla, en lugar de sobre todo el *dataset*.

11. Lean con atención el código a continuación antes de ir ejecutando paso a paso. Es importante que entiendan que está pasando en cada línea. Exploren qué hace `seq_range` experimentando en la consola.

```
# Creo una grilla de valores, con todos los valores de Petal.Length, pero
# solo cinco valores de la segunda variable
grilla <- data_grid(df, Petal.Length,
                    Sepal.Width = seq_range(Sepal.Width, n=5)) %>%
  add_predictions(mod2)

# Hago el gráfico, agrupando por valor de la segunda variable
df %>% ggplot(aes(x=Petal.Length, y=Petal.Width,
                  color=Sepal.Width)) +
  geom_point() +
  geom_line(data=grilla, aes(y=pred, group=Sepal.Width))
```

Para cada uno de los cinco valores de la segunda variable, tenemos una recta diferente. Fíjense que la pendiente de las rectas no cambia, solo la ordenada al origen.

12. Vuelvan a la expresión de la fórmula del modelo e intenten reconciliar este hecho con la expresión matemática. ¿Cómo queda el modelo si fijo el valor de la segunda variable predictora? ¿Podemos convertirlo en un modelo lineal “simple”? ¿Qué pasa si cambia el valor en el que fijamos a la segunda variable predictora? ¿Qué parte del modelo cambia?

Vamos a ver mayor flexibilidad cuando incluyamos modelos con términos de interacción, en un par de semanas.

i Nota

Una opción de visualización más sofisticada, que escapa a las habilidades que queremos que adquieran en esta materia, sería ver el gráfico en tres dimensiones. En este caso, el modelo es un **plano** en el espacio de los datos:

```
library(plotly)

x1.seq <- seq(min(df$Petal.Length),max(df$Petal.Length),length.out=25)
x2.seq <- seq(min(df$Sepal.Width),max(df$Sepal.Width),length.out=25)
z <- t(outer(x1.seq, x2.seq, function(x,y) coef(mod2)[1]+coef(mod2)[2]*x+coef(mod2)[3]*y))

plot_iris <- plot_ly(type='scatter3d', mode='markers',
                    data=df, x=~Petal.Length, y=~Sepal.Width, z=~Petal.Width,
                    marker=list(color='rgb(1, 1, 255)', size=5)) %>%
  # Add plane
  add_trace(type='surface',
            x=~x1.seq,
            y=~x2.seq,
            z =~z,
            opacity=0.7)

plot_iris
```

Esto está buenísimo, pero piensen qué pasaría si usáramos tres variables predictoras.

13. Para analizar los residuos de este modelo, aparece una nueva diferencia con el modelo simple. Hagan el gráfico de los residuos en función de cada una de las variables. ¿Observan algún patrón adicional?
14. Un gráfico que puede hacerse siempre, **independientemente de la cantidad de variables predictoras** que tenga el modelo, es el de los residuos en función de la predicción del modelo. Hagan este gráfico y discutan las diferencias con lo del punto anterior.

Una variable continua y una categórica

Ahora volvamos al *dataset* de Iris original y retengamos todas las especies.

15. Hagan un gráfico de la variable *objetivo* en función del largo del pétalo y colorean según la especie.
16. Hagan un modelo idéntico al del punto 1. Comparen los residuos y el R^2 .
17. Ahora incluyen a la especie de la flor como una variable descriptora adicional. Esto se hace **exactamente de la misma manera** en la sintaxis de `lm`: `Petal.Width ~ Petal.Length + Species`.

De nuevo, ojo con lo que significa realmente esto en término de la expresión matemática del modelo.

18. Obtengan el `summary` del modelo. ¿Cuántos parámetros tiene este modelo? ¿Cómo se compara con el modelo de dos variables continuas?
19. Hagan los gráficos correspondientes:
 - a. Residuos en función de las predicciones.
 - b. Predicciones del modelo en un gráfico de `Petal.Width` vs. `Petal.Length`, usando colores para cada especie.

Resumen de lo visto en esta Guía:

- Ajuste de modelos de regresión lineal simple y múltiple con `lm(y ~ x, data = ...)`.
- Interpretación de la salida de `summary(mod)`: coeficientes, error estándar, valor t y p valor.
- Cálculo de predicciones con `modelr::add_predictions()` y visualización del ajuste con `geom_line()` y `geom_abline()`.
- Cálculo de residuos con `modelr::add_residuals()`; gráfico de residuos para diagnosticar el modelo.
- Métricas de evaluación: RMSE y coeficiente de determinación R^2 con `modelr::rsquare()`.
- Modelos con múltiples variables explicativas (continuas y categóricas): fórmula y `~ x1 + x2`.
- Visualización de modelos multivariados con grillas de predicción usando `data_grid()` y `seq_range()`.

Bibliografía obligatoria

Libro de Wickham

Introducción a los modelos: <https://es.r4ds.hadley.nz/22-model.html>

Conceptos básicos de modelos: <https://es.r4ds.hadley.nz/23-model-basics.html>